

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Технології розподілених систем та паралельних обчислень»

Робота №5

Тема «Паралельні методи інтегрування. Використання функцій
колективного обміну MPI»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Дьячук Т.С.

МЕТА

Метою роботи є продовжити вивчення функцій обміну бібліотеки MPI, зокрема, функцій колективного обміну. Вивчити деякі прийоми їх використання для розподілення даних та обчислень між паралельними процесорами.

ЗАВДАННЯ

5.1.2.1 Розробити послідовні алгоритм та програму обчислення числа π з можливістю вимірювання часу роботи програми для заданого n та визначення помилки апроксимації інтеграла. Для цього використовуйте функцію вимірювання часу бібліотеки MPI, як еталонне значення числа π прийміть 3.141592653589793238462643.

Дослідіть точність та швидкодію програми обчислення π для значень кількості інтервалів 10^3 , 10^4 , 10^5 , 10^6 , 10^7 , 10^9 . Результати подайте у звіті у вигляді таблиці.

5.1.2.2 Реалізуйте власну паралельну програму обчислення інтеграла (1) з використанням наведеної програми 1, додайте до неї функцію обчислення часу її виконання. Визначте прискорення паралельної програми для чотирьох процесорів.

5.1.2.3 Використовуючи опановану техніку інтегрування, розробіть послідовні алгоритм та програму обчислення інтеграла у відповідності до наведених нижче варіантів. В програмі реалізуйте введення кількості інтервалів з клавіатури. Дослідіть точність та швидкодію програми для значень кількості інтервалів 10^3 , 10^4 , 10^5 , 10^6 , 10^7 , 10^9 . Результати подайте у звіті у вигляді таблиці.

Обчислити інтеграл

$$\int_0^{\frac{\pi}{2}} \sin^2(x) dx$$

5.2.2.1 Програму 5.2 доповніть функцією обчислення часу її виконання та функцією обчислення дисперсії σ^2 випадкової величини значення π .

$$\sigma^2 \approx \frac{1}{N-1} \sum_{i=1}^N (\xi_i - \bar{\xi}_N)^2$$

причому, оскільки середнє значення $\bar{\xi}_N$ заздалегідь не відомо, зручніше використати формулу:

$$\sigma^2 \approx \frac{1}{N+1} \left[\sum_{i=1}^N \xi_i^2 - \frac{1}{N} \left(\sum_{i=1}^N \xi_i \right)^2 \right]$$

5.2.2.2 Використовуючи формули (5.9) визначити N таке, щоб похибка ϵ обчислення числа π наближено дорівнювала 10^{-4} .

$$N \approx c\epsilon^{-2} \approx O(\epsilon^{-2})$$

5.2.2.3 Підібрати для знайденого N параметри N_in_proc та n_in_proc для випадку використання двох та чотирьох процесів і за допомогою доповненої програми обчислити число π , показати, що виконується відношення (5.8).

$$\epsilon \approx 3 \frac{\sigma}{\sqrt{N}}$$

5.2.2.4 Визначити прискорення паралельної програми для двох та чотирьох процесорів.

5.2.3.1 Розробити паралельну програму інтегрування методом Монте-Карло, використовуючи програму 5.3.

5.2.3.3 Виконати наступні варіанти завдання.

Розробити паралельну програму для обчислення кратного інтеграла $\iint_S f(x, y) dx dy$, де $f(x, y) = xy$, S – область, яка міститься між $x_1 = 0$, $x_2 = 1$ та $y_1(x) = x^2$, $y_2(x) = \sqrt{x}$ (точне значення інтеграла дорівнює $\frac{1}{12}$)

ВИКОНАННЯ

5.1.2.1

Завдання

Розробити послідовні алгоритм та програму обчислення числа π з можливістю вимірювання часу роботи програми для заданого n та визначення помилки апроксимації інтеграла. Для цього використовуйте функцію вимірювання часу бібліотеки MPI, як еталонне значення числа π прийміть 3.141592653589793238462643.

Дослідіть точність та швидкодію програми обчислення π для значень кількості інтервалів 10^3 , 10^4 , 10^5 , 10^6 , 10^7 , 10^9 . Результати подайте у звіті у вигляді таблиці.

Код

```
from mpi4py import MPI

def calculate_pi(n):
    step = 1.0 / n
    total_sum = 0.0

    start_time = MPI.Wtime()

    for i in range(n):
        x = (i + 0.5) * step
        total_sum += 4.0 / (1.0 + x * x)
    pi_approx = total_sum * step

    end_time = MPI.Wtime()

    return pi_approx, end_time - start_time

def main():
    PI_REF = 3.141592653589793238462643

    n_values = [10**3, 10**4, 10**5, 10**6, 10**7, 10**9]
```

```

    print(f"{'n':<12} | {'Calculated PI':<18} | {'Error':<21} | {'Time
(sec)':<12}")
    print("-" * 70)

    for n in n_values:
        pi_val, duration = calculate_pi(n)
        error = abs(PI_REF - pi_val)

        print(f"{'n':<12e} | {'pi_val':<18.15f} | {'error':<18.15e} |
{'duration':<12.6f}")

if __name__ == "__main__":
    main()

```

Таблиця

Значення N	Обчислений π	Помилка	Час роботи (сек)
10^3	3.141592736923123	8.333332957022321e-08	0.000097
10^4	3.141592654423134	8.333409518002100e-10	0.000945
10^5	3.141592653598162	8.368417070414580e-12	0.009637
10^6	3.141592653589764	2.886579864025407e-14	0.095930
10^7	3.141592653589731	6.217248937900877e-14	0.959065
10^9	3.141592653589971	1.776356839400250e-13	94.468141

5.1.2.2

Завдання

Реалізуйте власну паралельну програму обчислення інтеграла (1) з використанням наведеної програми 1, додайте до неї функцію обчислення часу її виконання. Визначте прискорення паралельної програми для чотирьох процесорів.

Код

```

from mpi4py import MPI
import math

```

```

def parallel_pi(N_total):
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    numprocs = comm.Get_size()

    n_local = N_total // numprocs

    comm.Barrier()
    start_time = MPI.Wtime()

    local_sum = 0.0
    start_index = rank * n_local
    end_index = (rank + 1) * n_local

    step = 1.0 / N_total
    for i in range(start_index, end_index):
        x = (i + 0.5) * step
        local_sum += 4.0 / (1.0 + x * x)

    total_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)

    end_time = MPI.Wtime()
    duration = end_time - start_time

    if rank == 0:
        pi_approx = total_sum * step
        return pi_approx, duration
    return None, duration

if __name__ == "__main__":
    N = 10**8

    pi_value, time_spent = parallel_pi(N)

    rank = MPI.COMM_WORLD.Get_rank()
    if rank == 0:
        pi_ref = math.pi
        print(f"Processes amount: {MPI.COMM_WORLD.Get_size()}")
        print(f"Calculated PI: {pi_value:.16f}")
        print(f"Error: {abs(pi_ref - pi_value):.16e}")
        print(f"Execution time: {time_spent:.6f} sec")

```

Результат

Вивід у консоль:

```

Processes amount: 4
Calculated PI: 3.1415926535896825
Error: 1.1057821325266559e-13
Execution time: 2.499878 sec

```

Для отримання прискорення потрібно:

1. запустити програму на одному потоці, отримати значення T_1 ;
2. запустити програму на чотирьох потоках, отримати значення T_4 ;
3. розрахувати прискорення, поділивши T_1 на T_4 .

Процес:

1. Значення $T_1 = 9.594051$;
2. Значення $T_4 = 2.499878$;
3. Значення S (прискорення) $= 3.83780768502$.

5.1.2.3

Завдання

Використовуючи опановану техніку інтегрування, розробіть послідовні алгоритм та програму обчислення інтеграла у відповідності до наведених нижче варіантів. В програмі реалізуйте введення кількості інтервалів з клавіатури. Дослідіть точність та швидкодію програми для значень кількості інтервалів 10^3 , 10^4 , 10^5 , 10^6 , 10^7 , 10^9 . Результати подайте у звіті у вигляді таблиці.

Обчислити інтеграл

$$\int_0^{\frac{\pi}{2}} \sin^2(x) dx$$

Код

```
from mpi4py import MPI
import numpy as np

def f(x):
    return np.sin(x) ** 2

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()
```

```

a = 0.0
b = np.pi / 2

if rank == 0:
    n = int(input("Please enter the number of intervals (n): "))
else:
    n = None

n = comm.bcast(n, root=0)

start_time = MPI.Wtime()

h = (b - a) / n
local_n = n // size
local_a = a + rank * local_n * h
local_b = local_a + local_n * h

integral = (f(local_a) + f(local_b)) / 2.0
for i in range(1, local_n):
    x = local_a + i * h
    integral += f(x)
integral *= h

total_integral = comm.reduce(integral, op=MPI.SUM, root=0)
end_time = MPI.Wtime()

if rank == 0:
    execution_time = end_time - start_time
    print(f"Result: {total_integral:.12f}")
    print(f"Execution time: {execution_time:.6f} sec")

if __name__ == "__main__":
    main()

```

Результати

```

Please enter the number of intervals (n): 3
Result: 0.785398163397
Execution time: 0.000630 sec

```

5.2.2.1

Завдання

5.2.2.1 Програму 5.2 доповніть функцією обчислення часу її виконання та функцією обчислення дисперсії σ^2 випадкової величини значення π .

$$\sigma^2 \approx \frac{1}{N-1} \sum_{i=1}^N (\xi_i - \bar{\xi}_N)^2$$

причому, оскільки середнє значення $\bar{\xi}_N$ заздалегідь не відомо, зручніше використати формулу:

$$\sigma^2 \approx \frac{1}{N-1} \left[\sum_{i=1}^N \xi_i^2 - \frac{1}{N} \left(\sum_{i=1}^N \xi_i \right)^2 \right]$$

Kod

```
import math
import random
import sys
import time
from mpi4py import MPI

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    numprocs = comm.Get_size()

    n_in_proc = 1_000_000
    R = 0.5
    pi_reference = math.pi

    n_iterations = 0
    if rank == 0:
        if len(sys.argv) > 1:
            n_iterations = int(sys.argv[1])
        else:
            n_iterations = 10

    n_iterations = comm.bcast(n_iterations, root=0)

    pi_values = []

    start_time = MPI.Wtime()

    for i in range(n_iterations):
        random.seed(time.time() + rank + i)

        in_circle_local = 0
        for _ in range(n_in_proc):
            x = random.uniform(-0.5, 0.5)
```

```

        y = random.uniform(-0.5, 0.5)
        if x * x + y * y < R * R:
            in_circle_local += 1

    total_in_circle = comm.reduce(in_circle_local, op=MPI.SUM,
root=0)

    if rank == 0:
        total_points_iter = n_in_proc * numprocs
        xi = 4.0 * (total_in_circle / total_points_iter)
        pi_values.append(xi)
        print(f"Iteration {i + 1}: pi = {xi:.10f}")

end_time = MPI.Wtime()

if rank == 0:
    n = len(pi_values)
    sum_xi = sum(pi_values)
    sum_xi_sq = sum(x**2 for x in pi_values)

    pi_final = sum_xi / n

    variance = (1.0 / (n + 1)) * (sum_xi_sq - (1.0 / n) *
(sum_xi**2))

    execution_time = end_time - start_time

    print("--- Results ---")
    print(f"Amount of iterations (N): {n}")
    print(f"Dots per iteration:      {n_in_proc * numprocs}")
    print(f"Final PI (mean):           {pi_final:.16f}")
    print(f"Error:                      {abs(pi_reference -
pi_final):.16f}")
    print(f"Variance (sigma^2):       {variance:.16e}")
    print(f"Execution time:           {execution_time:.6f} sec")

if __name__ == "__main__":
    main()

```

Результати

```

Iteration 1: pi = 3.1404240000
Iteration 2: pi = 3.1390520000
Iteration 3: pi = 3.1390280000
Iteration 4: pi = 3.1419760000
Iteration 5: pi = 3.1400280000
Iteration 6: pi = 3.1403280000
Iteration 7: pi = 3.1401840000
Iteration 8: pi = 3.1450040000
Iteration 9: pi = 3.1402640000
Iteration 10: pi = 3.1404440000
--- Results ---
Amount of iterations (N): 10
Dots per iteration:      1000000

```

Final PI (mean):	3.1406732000000002
Error:	0.0009194535897930
Variance (sigma^2):	2.4404481448217750e-06
Execution time:	3.435761 sec

5.2.2.2

Завдання

Використовуючи формули (5.9) визначити N таке, щоб похибка ε обчислення числа π наближено дорівнювала 10^{-4} .

$$N \approx c\varepsilon^{-2} \approx O(\varepsilon^{-2})$$

Код

```
import math
import random
import time
from mpi4py import MPI

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    epsilon = 1e-4
    # N = pi*(4-pi) / epsilon^2
    total_n = int(2.697 * (epsilon**-2))

    local_n = total_n // size
    if rank == size - 1:
        local_n += total_n % size

    random.seed(time.time() + rank)
    count = 0

    start_time = MPI.Wtime()

    for _ in range(local_n):
        x = random.random()
        y = random.random()
        if x * x + y * y <= 1.0:
            count += 1

    total_count = comm.reduce(count, op=MPI.SUM, root=0)
    end_time = MPI.Wtime()
```

```

if rank == 0:
    pi_approx = 4.0 * total_count / total_n
    actual_error = abs(math.pi - pi_approx)

    print(f"--- Results for epsilon = {epsilon} ---")
    print(f"Total number of dots (N): {total_n}")
    print(f"Calculated PI: {pi_approx:.10f}")
    print(f"Actual error: {actual_error:.10e}")
    print(f"Execution time for {size} processors: {end_time -
start_time:.4f} sec")

if __name__ == "__main__":
    main()

```

Результати

```

--- Results for epsilon = 0.0001 ---
Total number of dots (N): 269699999
Calculated PI: 3.1414958811
Actual error: 9.6772457068e-05
Execution time for 12 processors: 5.9276 sec

```

5.2.2.3

Завдання

Підібрати для знайденого N параметри N_in_proc та n_in_proc для випадку використання двох та чотирьох процесів і за допомогою доповненої програми обчислити число π , показати, що виконується відношення (5.8).

$$\epsilon \approx 3 \frac{\sigma}{\sqrt{N}}$$

Код

```

import math
import random
import time
from mpi4py import MPI

```

```

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    if size == 2:
        n_iterations = 135
    elif size == 4:
        n_iterations = 67
    else:
        n_iterations = 100

    n_in_proc = 1_000_000
    R = 0.5

    pi_values = []
    start_time = MPI.Wtime()

    for i in range(n_iterations):
        random.seed(time.time() + rank + i)
        in_circle_local = 0
        for _ in range(n_in_proc):
            x = random.uniform(-0.5, 0.5)
            y = random.uniform(-0.5, 0.5)
            if x * x + y * y ≤ R * R:
                in_circle_local += 1

        total_in_circle = comm.reduce(in_circle_local, op=MPI.SUM,
root=0)

        if rank == 0:
            points_this_iter = n_in_proc * size
            xi = 4.0 * (total_in_circle / points_this_iter)
            pi_values.append(xi)

    end_time = MPI.Wtime()

    if rank == 0:
        N_total = n_iterations * n_in_proc * size
        sum_xi = sum(pi_values)
        sum_xi_sq = sum(x**2 for x in pi_values)

        pi_final = sum_xi / n_iterations
        variance = (1.0 / (n_iterations + 1)) * (
            sum_xi_sq - (1.0 / n_iterations) * (sum_xi**2)
        )
        sigma = math.sqrt(abs(variance))

        actual_error = abs(math.pi - pi_final)
        theoretical_limit = 3 * (sigma / math.sqrt(n_iterations))

        print(f"--- Results for {size} processors ---")
        print(f"General N: {N_total}")
        print(f"Calculated PI: {pi_final:.10f}")
        print(f"Actual error: {actual_error:.10e}")

```

```

        print(f"Theroretical limit (3*sigma/sqrt(N_iter)):\n{theoretical_limit:.10e}")
        print(f"Relationship (5.8) succeding: {actual_error <= theoretical_limit}")
        print(f"Execution time: {end_time - start_time:.4f} сек")

if __name__ == "__main__":
    main()

```

Результати

```

--- Results for 1 processors ---
General N: 100000000
Calculated PI: 3.5804846800
Actual error: 4.3889202641e-01
Theroretical limit (3*sigma/sqrt(N_iter)): 3.6944752100e-04
Relationship (5.8) succeding: False
Execution time: 34.8550 сек

```

5.2.2.4

Завдання

Визначити прискорення паралельної програми для двох та чотирьох процесорів.

Виконання

Для виконання обрано програму 5.1.2.3. Її запуснено на двох та чотирьох процесорах.

Для обчислення прискорення було використано формулу $S_p = T_1 / T_p$.

Таблиця

Кількість процесорів, p	Час виконання, T _p	Прискорення, S
1	0.000225	1

2	0.000440	0,511363636
4	0.000832	0,270432692

5.2.3.1

Завдання

Розробити паралельну програму інтегрування методом Монте-Карло, використовуючи програму 5.3.

Код

```
import math
import random
import time
from mpi4py import MPI

def f(x):
    return math.sin(x) ** 2

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    numprocs = comm.Get_size()

    a = 0.0
    b = math.pi / 2
    N_total = 1_000_000_000
    n_per_procs = N_total // numprocs

    reference = math.pi / 4
    local_sum = 0.0

    start_time = MPI.Wtime()

    random.seed(time.time() + rank)

    for _ in range(n_per_procs):
        x = random.uniform(a, b)
        local_sum += f(x)

    total_sum = comm.reduce(local_sum, op=MPI.SUM)
    end_time = MPI.Wtime()

    if rank == 0:
        integral_mc = ((b - a) * total_sum) / N_total
```

```

    error = abs(reference - integral_mc)

    print(f"Result: {integral_mc:.16f}")
    print(f"Accurate number: {reference:.16f}")
    print(f"Error: {error:.15f}")
    print(f"Execution time: {end_time - start_time:.4f} sec")

if __name__ == "__main__":
    main()

```

Результати

```

Result: 0.7854164765012532
Accurate number: 0.7853981633974483
Error: 0.000018313103805
Execution time: 53.4747 sec

```

5.2.3.3

Завдання

Виконати наступні варіанти завдання.

Розробити паралельну програму для обчислення кратного інтеграла $\iint_S f(x, y) dx dy$, де $f(x, y) = xy$, S – область, яка міститься між $x_1 = 0$, $x_2 = 1$ та $y_1(x) = x^2$, $y_2(x) = \sqrt{x}$ (точне значення інтеграла дорівнює $\frac{1}{12}$)

Код

```

import math
import random
import time
from mpi4py import MPI

def f(x, y):
    return x * y

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    numprocs = comm.Get_size()

    N_total = 100_000_000
    n_per_procs = N_total // numprocs

    x_min, x_max = 0.0, 1.0
    y_min, y_max = 0.0, 1.0
    area_rect = (x_max - x_min) * (y_max - y_min)

```



```

local_sum = 0.0
points_in_S = 0

random.seed(time.time() + rank)

start_time = MPI.Wtime()

for _ in range(n_per_procs):
    x = random.uniform(x_min, x_max)
    y = random.uniform(y_min, y_max)

    if x**2 ≤ y ≤ math.sqrt(x):
        local_sum += f(x, y)
        points_in_S += 1

total_f_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)
total_points_in_S = comm.reduce(points_in_S, op=MPI.SUM, root=0)
end_time = MPI.Wtime()

if rank == 0:
    # I = Area_rect * (Sum_f_in_S / N_total)
    integral_mc = (area_rect * total_f_sum) / N_total

    reference = 1.0 / 12.0
    error = abs(reference - integral_mc)

    area_S_approx = (area_rect * total_points_in_S) / N_total

    print("--- Calculated double integral ---")
    print(f"Number of processes: {numprocs}")
    print(f"Total number of points N: {N_total}")
    print(f"Approcimated area of area S: {area_S_approx:.6f}
(accurate: 1/3)")
    print(f"Integration results: {integral_mc:.10f}")
    print(f"Accurate value (1/12): {reference:.10f}")
    print(f"Absolute error: {error:.10e}")
    print(f"Execution time: {end_time - start_time:.4f} sec")

if __name__ == "__main__":
    main()

```

Результати

```

--- Calculated double integral ---
Number of processes: 12
Total number of points N: 100000000
Approcimated area of area S: 0.333378 (accurate: 1/3)
Integration results: 0.0833352054
Accurate value (1/12): 0.0833333333
Absolute error: 1.8720723393e-06
Execution time: 6.9551 sec

```

ПИТАННЯ

Проаналізуйте програму 5.1. Нехай інтервал інтегрування складається з 10 підінтервалів (0, 1, 2 і т.д.), а кількість процесорів дорівнює чотирьом. Назвіть номери підінтервалів, в яких обчислює підінтегральну функцію кожний з процесорів.

Якщо маємо за $n = 10$ інтервалів та $size = 4$ процесори, тоді інтервали розподіляються так:

- процесор 0: інтервали 1, 5, 9;
- процесор 1: інтервали 2, 6, 10;
- процесор 2: інтервали 3, 7;
- процесор 3: інтервали 4, 8.

Чи залежить прискорення обчислення числа π за допомогою програми 1 від кількості підінтервалів?

Так, залежить.

Порівняйте точності обчислень числа π за допомогою програм 5.1 та 5.3. Які ваші висновки щодо точності реалізованих в програмах 5.1 та 5.3 методів?

Було проведено обчислення на двох програмах.

Результати першої: 3.1508492098656036.

Результати другої: 3.1416110770196730.

Похибка першої: 0.0092565562758105.

Похибка другої: 0.000018423429880.

З цього видно, що друга програма обчислює число π більш точно. Але вона і вимагає більше часу для виконання.

Як в MPI визначити розмір буфера, необхідного для прийому повідомлень?

У бібліотеці `mpi4py` для малих об'єктів розмір визначається автоматично через серіалізацію, а для великих використовується `nbytes` об'єкти `NumPy`.

Що таке групи процесів в MPI? За допомогою яких функцій бібліотеки MPI їх можна створити?

Група процесів в MPI є впорядкованою множиною процесів.

Основні функції для роботи з групами:

- `MPI_Comm_group` – отримати групу з комунікатора;
- `MPI_Group_incl` – створити групу з вибраних рангів;
- `MPI_Group_excl` – створити групу, виключивши певні ранги;
- `MPI_Group_union` / `MPI_Group_intersection` – операції над множинами.

Як здійснити обмін даними між процесами групи?

Обмін даних здійснюється через комунікатори.

Кроки виконання:

1. створюється нова група;
2. на основі групи створюється новий комунікатор:
`MPI_Comm_create(old_comm, group, &new_comm)`.
3. обмін (`send/recv`, `bcast`) відбувається через новий комунікатор.

Поясніть призначення і роботу функції

**MPI_Comm_split(MPI_Comm comm, int split, int key, MPI_Comm
*newcomm);**

**Чи можуть декілька комунікаторів в одній паралельній
програмі мати однакові ідентифікатори?**